Correct Optimizations: Phase 2 Report Based on "Minotaur: A SIMD-Oriented Synthesizing Superoptimizer" [1]

Kilian Schulz
Technical University Munich

Ilja Gamza
Technical University Munich

Tomás Gaspar
Technical University Munich

1 Introduction

Minotaur [1] is a superoptimizer, that after computing optimizations, caches those in a data base. These optimizations consist of a key, which is the original code, and a value, which is an equivalent, but faster, rewrite of the original code. This caching is done in order to reduce the substantial runtimes of Minotaur for future compilations of the same or a similar program. Minotaurs compile times, with a cold cache, can easily be hundreds of time slower than with regular LLVM [1].

Currently Minotaur directly uses the extracted code segments in LLVM IR as the keys and stores those along side a rewrite, in Minotaurs own IR as value, in the cache as shown in example 1.

This approach can easily lead to near misses, where an equivalent code segment is already cached, but for instance has unused arguments, leading to a cache miss and a costly recompute of an optimization.

One way to avoid this issue is to attempt to increase the number of cache hits while keeping the correctness of the mapping between keys and their rewrite. This leads to reduced compile times of Minotaur even when starting from a cold cache.

2 Research Proposal

Our solution introduces an additional step after the cut extraction. It performs multiple code transformations, with the goal of reducing the number of possible equivalent keys, mapping those to one canonicalized key.

The idea is that this leads to fewer cache entries, increasing the cache hit rate and reducing the number of slow optimization searches by Minotaur. For large enough projects this can already take effect during a build even when starting with an empty cache thereby reducing compile time.

3 Canonicalization

Our extended version of Minotaur is available on GitHub [1].

The canonicalizer is placed as a pre- and postprocessing step around Minotaurs inference step. We intercept the LLVM IR produced by the slicer and perform transformations on it to produce a canonicalized version of the input which is then forwarded to the inference step. In the following this transformation is called canonicalization. After the inference step is finishes and returns a rewrite (in Minotaurs own IR), the canoncializer again performs transformations on the rewrites. This process is called de-canonicalization. The structure of the augmented Minotaur with our canonicalizer is shown in figure 2.

Each canonicalization-step in our implementation consists of a canonicalization function and a de-canonicalization function. The canonicalization-steps are ordered as to not interfere with each other. More on that is discussed in section 3.2. During the canonicalization process every canonicalization function is applied iteratively in this predefined order. For de-canonicalization the de-canonicalization functions are applied in the reverse order in which the canonicalization-steps are specified. It is also possible to pass information between the pairs of canonicalization and de-canonicalization functions of each step if required. This enables the support for more complex transformations.

3.1 Canonicalization Steps

In the following we explain each canonicalization-step and show its correctness. We show for each canonicalization step that applying it yields an equivalent function that has the same depth as the original function. In this case two functions are considered equivalent iff appling them to the same arguments yield an equal output (and changes to memory). The depth is kept constant as increasing this has large impact on runtime as shown in original paper [1].

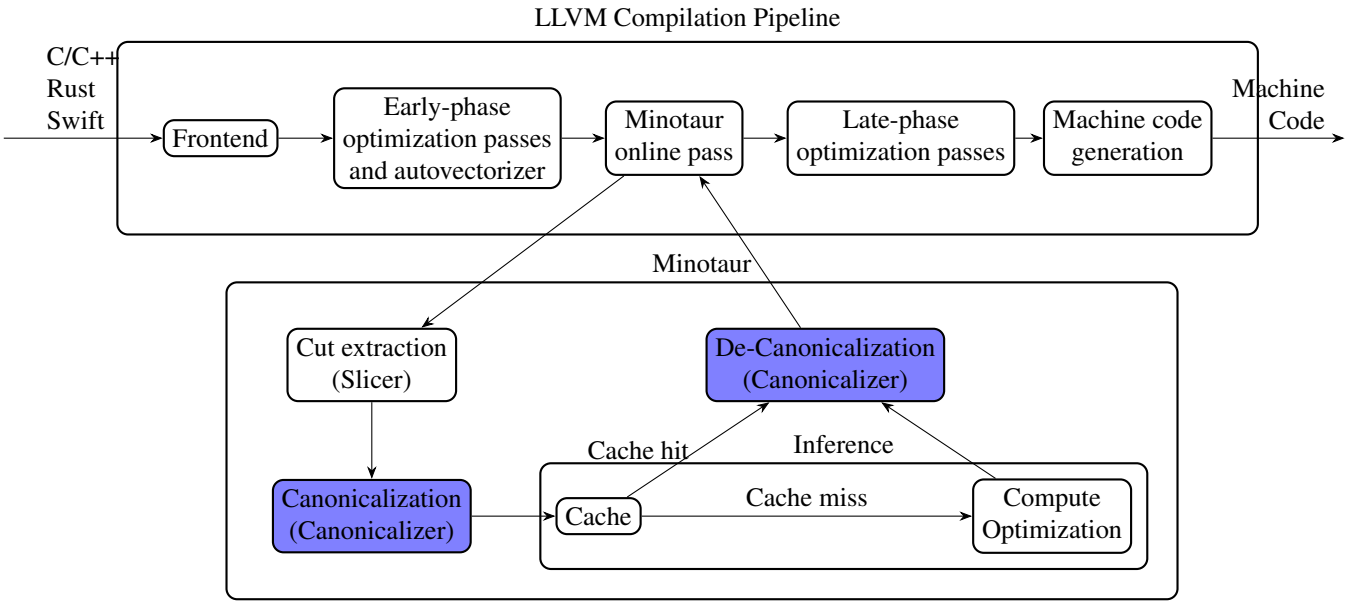
[1] Extended Minotaur Version

Figure 1: Example of a key value pair as it is found in the Minotaur cache.

```
key:
        define i1 @cut(i64 %__n2, i16 %__n3) {
                %__n0 = sub nsw i64 0, %__n2
                %__n1 = icmp samesign ult i64 %__n0, 2033
                ret i1 %__n1
        }

value:
        (icmp_sgt (var i64 %__n2) (reservedconst i64 |i64 -2033|) b1)
```

Figure 2: Structure of Minotaur and how the Canonicalization is integrated. The highlighted blocks were added for (De-)Canonicalization.



An example of all the steps applied sequentially to an example input is shown in figure 3

Elimination of Unused Function Arguments Some function have unused arguments. Those can simply be eliminated during the forward pass. Assuming the given input function f has an unused argument we can define the following. Let $f : T_1 \times ... \times T_i \times ... \times T_n \to R$ and $g : T_1 \times ... \times T_{i-1} \times T_{i+1} \times ... \times T_n \to R$ be n-ary and $n-1$-ary functions respectively with

$$f(t_1,...,t_n) := g(t_1,...,t_{i-1},t_{i+1},...,t_n), t_j \in T_j$$

Since t_i is not part of the function signature of g we can define a new function $f' := g$. This obviously does not change what is computed since

$$\begin{aligned} f(t_1,...,t_n) &= g(t_1,...,t_{i-1},t_{i+1},...,t_n) \\ &= f'(t_1,...,t_{i-1},t_{i+1},...,t_n) \end{aligned}$$

Due to the implicit nature of the function signature in the values of the cache and the structure of Minotaur no additional work is required in relation to this step.

Sorting of Function Arguments Another modification that is done to the function signature is that we sort the arguments. The arguments are sorted by type-name and internally by when they are used inside the function body. Sorting by the type name is done in order to avoid cases where the function body is the same but a cache miss occurs due to the order of the types being different. To achieve a total order on the function arguments, each set of arguments of the same type are

sorted by when they are used in the function body. This makes the argument order dependent on the function body and has to be taken into account when ordering the canonicalization steps. The correctness of the step is shown in the following. Let $f : T_1 \times ... \times T_n \to R$ be the input function and $f' : T_{\rho(1)} \times ... \times T_{\rho(n)} \to R$ the output function with $\rho : [n] \to [n]$ being a bijective function i.e. a permutation. We define

$$\begin{aligned} f'(t_{\rho(1)}, ..., t_{\rho(n)}) &:= f(t_{\rho^{-1}(\rho(1))}, ..., t_{\rho^{-1}(\rho(n))}) \\ &= f(t_1, ..., t_n), t_i \in T_i \end{aligned}$$

This clearly shows that f and f' compute the same for all inputs. Similar to the elimination of unused arguments no further work is required.

Renaming of Function Arguments

Replacing Greater-Than with Less-Than

Non-strict Operator Replacement The idea is to reduce the number of comparison operators that are available. This is done by replacing non-strict comparison operators like $\leq$ with $<$. To introduce additional operations this is only done when a constant is one of the operants. This constant is then adjusted accordingly if no prevented by an edge case like the constant being already the maximum or minimum value possible for the data type. So given some expression like $x \leq c$ it would be transformed into $x < c + \varepsilon$ where ε is the smallest possible increment possible for c.

$$\exists v \exists \varepsilon \forall x. ((v = c + \varepsilon) \implies (x \leq c \iff x < c + \varepsilon))$$

An analogous argument can be made for the $\geq$ and for a swapped operands and works for both integral and floating point types. Currently vector types are filtered.

3.2 Ordering of the Canonicalization Steps

3.3 Issues with the original version

4 Evaluation

4.1 Setup

4.2 Results

The key metrics which measured for the following benchmarks were the cache size, cache hit rate, number of applied rewrites and solver time. Since our canonicalization steps are compositional we measure the impact of each step and combination of steps. We also tested changing the order of canonicalization steps where possible and measured the effect. Furthermore, we measure the total compilation time for several select software projects.

5 Discussion

6 Future Work

7 Conclusion

References

[1] Zhengyang Liu, Stefan Mada, and John Regehr. "Minotaur: A SIMD-Oriented Synthesizing Superoptimizer". In: *Proceedings of the ACM on Programming Languages (OOPSLA2)* 8.OOPSLA2 (Oct. 2024). https://doi.org/10.1145/3689766, pp. 1–25. DOI: 10.1145/3689766.

Figure 3: Example applying each canonicalization step.

```
input:
        define i1 @cut(i32 %a, float %b, i32 %c, float %d) {
                %1 = fcmp uge float %b, %d
                %2 = icmp sle i32 %c, 15

                %3 = and i1 %1, %2

                ret i1 %3
        }

unused argument elimination:
        define i1 @cut (float %b, i32 %c, float %d) {
                %1 = fcmp uge float %b, %d
                %2 = icmp sle i32 %c, 15

                %3 = and i1 %1, %2

                ret i1 %3
        }

replacing greater-than with less-than:
        define i1 @cut(float %b, i32 %c, float %d) {
                %1 = fcmp ule float %d, %b
                %2 = icmp sle i32 %c, 15

                %3 = and i1 %1, %2

                ret i1 %3
        }

replacing non-strict operators:
        define i1 @cut(float %b, i32 %c, float %d) {
                %1 = fcmp ule float %d, %b
                %2 = icmp slt i32 %c, 16

                %3 = and i1 %1, %2

                ret i1 %3
        }

argument sorting:
        define i1 @cut(float %d, float %b, i32 %c) {
                %1 = fcmp uge float %d, %b
                %2 = icmp sle i32 %c, 15

                %3 = and i1 %1, %2

                ret i1 %3
        }

argument renaming:
        define i1 @cut(float %__canonArg, float %__canonArg1,
                i32 %__canonArg2, i32 %__canonArg3) {
                %1 = fcmp uge float %__canonArg, %__canonArg1
                %2 = icmp sge i32 %__canonArg2, %__canonArg3
        }
```